PROJECT ARCHITECTURE · IMPLEMENTATION PLAN

Axiom

Version 2

Expanding Axiom from a load-testing API client into a complete, multi-functional mobile QA testing suite — covering functional, workflow, performance, resilience, security, and regression testing.

SCOPE

11 Phases · 11 Testing Types

FOUNDATION

Builds on the Existing Axiom v1 Codebase — No Rewrites

PREPARED

July 2026 · Version 2.0 · Draft for Review

Overview

Axiom v1 shipped a working mobile API client with request execution, response inspection, and batched load testing with percentile latency reporting (p50/p95/p99). This plan extends that foundation into a complete QA tool covering the major categories of API testing a development team needs — functional correctness, multi-step workflows, cross-environment consistency, performance under varied load shapes, resilience, security hygiene, and regression detection.

Every phase below is designed to extend existing Axiom services (networkService.ts, benchmarkEngine.ts, variableService.ts, environmentStore.ts, dataService.ts, and the Supabase schema) rather than introduce a parallel architecture. Phases are ordered so that each one unlocks or strengthens the phases after it.

Guiding Principle

Assertions come first because nearly everything else — the Collection Runner, regression testing, contract testing, security checks, and idempotency testing — depends on having a pass/fail evaluation layer. Building it once, early, avoids re-deriving pass/fail logic separately for every later feature.

Implementation Roadmap

Eleven phases, sequenced by dependency — earlier phases unblock later ones.

Phase	Feature	Testing Type Unlocked	Builds On
1	**Assertions Engine**	Functional Testing	networkService.ts response object
2	**Collection Runner & Variable Chaining**	End-to-End / Workflow Testing	environmentStore.ts, variableService.ts
3	**Environment Matrix Testing**	Cross-Environment Regression	environmentStore.ts, Collection Runner
4	**Stress / Spike / Soak Modes**	Advanced Performance Testing	benchmarkEngine.ts
5	**Regression & Contract Testing**	Schema / Baseline Diffing	history.response_body (Supabase)
6	**Fuzz & Negative Testing**	Input Robustness Testing	benchmarkEngine.ts batching pattern
7	**Security Smoke Testing**	Header / TLS / Auth-Boundary Checks	networkService.ts, assertion engine
8	**Network Condition (Chaos) Testing**	Resilience / Timeout Testing	executeRequest() abort/timeout logic
9	**Idempotency Testing**	Side-Effect Consistency Testing	assertion engine, diff logic
10	**Node Proxy Scheduler & Synthetic Monitoring**	Uptime / Health Monitoring	Planned Node.js/Express proxy (Phase 4, v1 plan)
11	**CI / Export Capability**	Team & Pipeline Integration	networkService.ts, assertion engine (portable TS)

PHASE 1

Assertions Engine

The foundation — pass/fail evaluation for every request Axiom sends

Right now Axiom captures rich response data (status, latency, TTFB, size, body) but has no way to programmatically judge whether a response was "correct." This phase adds that judgment layer, turning every request — single or batched — into a real test.

What to Build

- **Operator-based assertion model:** instead of eval()-ing raw JavaScript on-device (unsafe on mobile, hard to sandbox), store assertions as structured rows: { field, operator, expectedValue }. Support operators: equals, contains, exists, matchesRegex, lessThan / greaterThan (for status/latency), and jsonPathEquals for nested body fields.
- **New service — assertionEngine.ts:** takes the ResponseTiming object already returned by executeRequest() and evaluates the configured assertions against it, returning { passed: boolean, failures: string[] }.
- **"Tests" tab in the request UI:** a simple form to add/edit assertions per saved request, reusing the KeyValueEditor-style pattern already used for headers/params.
- **Wire into the benchmark loop:** call the assertion engine inside runSingleIteration() in benchmarkEngine.ts, so every load test iteration is also a functional check — not just a latency measurement.
- **Schema addition:** an assertions table (request_id, field, operator, expected_value), plus a passed boolean and failures JSONB column on history and benchmark_iterations.

Architecture Fit

Existing Piece	How It's Used / Extended
request.pre_request_script / post_response_script	Already reserved fields in the Request type — repurpose post_response_script as the natural home for assertion config, or introduce the dedicated assertions table (recommended for a structured UI over raw script editing).
networkService.ts → executeRequest()	No changes needed. The ResponseTiming object it already returns is exactly what assertionEngine.ts needs to evaluate.
benchmarkEngine.ts → runSingleIteration()	Add one call to the assertion engine after executeRequest() resolves; extend BenchmarkIteration with passed/failures.

PHASE 2

Collection Runner & Variable Chaining

The single highest-value feature — turns single requests into real QA workflows

Most real API bugs live in multi-step flows (login → create → fetch → delete → verify), not single isolated calls. This is the feature that makes Axiom a workflow tester rather than a request replayer, and it is the core value proposition of tools like Postman's Collection Runner.

What to Build

- **Sequential collection execution:** run every request inside a collection() in a defined order (respecting sort_order, already in your schema), rather than one request at a time.
- **Response-to-variable extraction:** let a user mark a JSONPath in response A's body (e.g. data.token or data.id) to be written into the active environment's variables at runtime — reusing the interpolation pipeline variableService.ts already implements for {{variable}} syntax.
- **Runner UI:** a new screen/tab showing each step of the run, its status (pending/running/pass/fail), and per-step latency and assertion results (from Phase 1).
- **Run persistence:** a collection_runs table (mirroring the benchmark_runs pattern already in your schema) storing per-run, per-step outcomes for later review.

Architecture Fit

Existing Piece	How It's Used / Extended
environmentStore.ts + variableService.ts	Chaining is implemented as: after each step, run the extraction rule, then call the same setVariable-style logic your environment store already exposes. No new interpolation engine required.
collectionsStore.ts	Already models the collection → requests relationship and sort_order needed to define run sequence.
assertionEngine.ts (Phase 1)	Each step's pass/fail comes directly from the assertion engine built in Phase 1 — this is why assertions had to come first.

PHASE 3

Environment Matrix Testing

Catch "works in staging, breaks in production" automatically

Environments are already first-class in Axiom's data model, which makes this a comparatively low-cost, high-differentiation feature — most API clients require manually switching environments and re-running by hand.

What to Build

- **Multi-environment execution:** run a single request, or a full collection (Phase 2), against two or more selected environments in one action.
- **Side-by-side diff view:** compare status code, latency, and body shape across environments in one screen, flagging any mismatch automatically.
- **Matrix history:** persist matrix runs so environment drift can be tracked over time, not just observed in the moment.

Architecture Fit

Existing Piece	How It's Used / Extended
environmentStore.ts	Already manages the list of environments and their variables — this phase adds "select several" instead of "select one active" and loops execution per environment.
Collection Runner (Phase 2)	Reused as the execution engine per environment; the matrix view is a presentation layer on top of N runner executions.

PHASE 4

Stress / Spike / Soak Modes

Extending the existing load engine — not rebuilding it

Axiom's benchmarkEngine.ts already performs fixed-load testing with percentile reporting. Stress, spike, and soak are different load shapes on the same batching loop, not new engines.

What to Build

- **Stress mode:** ramp batchSize upward across successive batches until the error rate crosses a configurable threshold (e.g. >5% non-2xx), reporting the breaking point rather than a fixed-load summary.
- **Spike mode:** a single oversized burst batch inserted into an otherwise steady run, with recovery latency tracked in the batches immediately after.
- **Soak mode:** low, sustained batches over a longer duration using setInterval rather than a tight loop, paired with expo-keep-awake to prevent the run from being suspended by screen lock. Foreground-only for now — reliable unattended, long-duration soak testing depends on the Phase 10 backend scheduler.
- **Mode selector:** add mode: 'fixed' | 'ramp' | 'spike' | 'soak' to BenchmarkRunOptions and branch the per-batch size calculation accordingly.

Architecture Fit

Existing Piece	How It's Used / Extended
benchmarkEngine.ts	All four modes reuse runSingleIteration(), the batching loop, and the existing p50/p95/p99 aggregation — only the batch-size-per-iteration calculation changes per mode.
benchmark_runs schema	Add a mode column; existing min/max/avg/percentile/error_rate columns apply unchanged to every mode.

PHASE 5

Regression & Contract Testing

Diff-based testing — build once, reuse for two use cases

Both regression and contract testing are fundamentally the same operation — comparing a response against an expected shape — so they should share one diff engine with two entry points.

What to Build

- **Baseline pinning:** let a user mark a saved history.response_body (already stored in full as TEXT in your schema) as the "golden" baseline for a request.
- **Structural JSON diff:** a diff utility that compares by key/value structure rather than raw text, so key reordering or whitespace doesn't produce false positives.
- **Contract mode:** either paste a JSON Schema or auto-infer one from a saved baseline response, then validate future runs' shape (types, required fields) against it — reusing the same diff engine with a schema-shaped comparison instead of a value-shaped one.
- **Regression alerts:** surface a clear pass/fail plus a readable diff summary in the response panel when a run deviates from its pinned baseline.

Architecture Fit

Existing Piece	How It's Used / Extended
history.response_body	Already persisted in full for every executed request — no new storage needed to support baseline pinning.
assertionEngine.ts (Phase 1)	"Matches baseline" and "matches schema" become two more operator types the assertion engine can evaluate.

PHASE 6

Fuzz & Negative Testing

Confirm the API degrades gracefully, not catastrophically

Fuzzing intentionally sends malformed input and checks that the API responds with a controlled 4xx rather than crashing with a 500 or hanging — a common, real source of production incidents.

What to Build

- **New service — fuzzEngine.ts:** takes a saved request's params/body and mutates one field per iteration: null, wrong type, oversized string, empty string, unicode edge cases, or a missing required field.
- **Mutation strategies:** start with a small, well-understood library of mutation types rather than fully random fuzzing, so results stay explainable ("this failed because we sent null for a required field").
- **Result grouping:** aggregate by which mutation caused a 5xx or timeout, rather than by latency stats — this is a correctness report, not a performance report.

Architecture Fit

Existing Piece	How It's Used / Extended
benchmarkEngine.ts batching pattern	fuzzEngine.ts reuses the same batch-and-collect loop almost directly — only the per-iteration request construction changes (mutated payload instead of identical payload).
networkService.ts → executeRequest()	Unchanged — mutated requests are executed exactly like normal ones.

PHASE 7

Security Smoke Testing

Lightweight automated checks, scoped to the user's own APIs

Positioned deliberately as QA hygiene checks for APIs the user owns and controls — not a generic scanning tool — both for responsible use and to avoid raising concerns during app store review.

What to Build

- **Header hygiene checks:** flag missing Content-Security-Policy, X-Content-Type-Options, Strict-Transport-Security, and similar headers on responses.
- **TLS enforcement check:** confirm an http:// variant of a request either fails or redirects to https://, rather than succeeding in plaintext.
- **Auth-boundary check:** re-run a saved request with its Authorization header stripped and assert the response is 401/403, not 200 — a simple, high-value check most API clients don't offer at all.
- **Deep TLS inspection (optional, proxy-assisted):** cipher suite and certificate chain detail isn't available to plain fetch() in React Native — route this specific check through the Phase 10 Node proxy, which can use Node's tls module properly.

Architecture Fit

Existing Piece	How It's Used / Extended
assertionEngine.ts (Phase 1)	Header presence and status-code boundary checks are just more assertion types, not a new evaluation system.
Node proxy (Phase 10)	Only needed for the deep TLS/certificate check; header and auth-boundary checks work entirely on-device today.

PHASE 8

Network Condition (Chaos) Testing

A mobile-native testing type desktop tools can't meaningfully offer

True packet-level throttling isn't available from JavaScript, but delay-injection and forced-abort are enough to validate how gracefully an app-under-test's retry and timeout logic behaves — a genuine mobile QA pain point.

What to Build

- **Delay injection:** artificially hold a request for a configured duration before allowing it to fire, to simulate a slow network.
- **Forced abort percentage:** randomly abort a configurable percentage of requests in a run to simulate flaky connectivity, and observe how the client-under-test handles it.
- **Timeout validation:** assert that a deliberately delayed request is correctly abandoned by the client-under-test's own timeout logic, rather than hanging indefinitely.

Architecture Fit

Existing Piece	How It's Used / Extended
networkService.ts → executeRequest()	Already implements AbortController-based timeout and cancellation — delay/abort injection wraps this existing logic rather than replacing it.

PHASE 9

Idempotency Testing

Catch a common, real bug class: non-idempotent writes

Fires the same write request (POST/PUT) twice in immediate succession and compares the two responses — flagging APIs that produce different side effects or state on a repeat call when they shouldn't.

What to Build

- **Repeat-and-compare execution:** run a saved write request twice back to back and capture both responses.
- **Field-level tolerance config:** let the user mark which response fields are expected to vary between calls (timestamps, generated IDs) versus which must be identical, so the comparison doesn't produce false positives.
- **Pass/fail via the diff engine:** reuse the structural diff logic from Phase 5, applied to the two responses instead of a response-vs-baseline comparison.

Architecture Fit

Existing Piece	How It's Used / Extended
Diff engine (Phase 5)	Directly reused — idempotency testing is "diff response A against response B" instead of "diff response against a pinned baseline."
assertionEngine.ts (Phase 1)	Produces the final pass/fail and failure summary shown to the user.

PHASE 10

Node Proxy Scheduler & Synthetic Monitoring

Moving what mobile can't do reliably off the device

Plain Expo managed apps suspend JS execution within roughly 30 seconds of being backgrounded, and iOS/Android impose their own limits even with background task APIs. Long-duration soak testing and "check every N minutes even when the app is closed" monitoring both need a server-side clock to be reliable — this phase builds the Node.js proxy already scoped in the original v1 plan, now with a scheduling role.

What to Build

- **Node.js/Express proxy:** as originally scoped, forwards requests server-side and returns raw responses — useful for auth-header-sensitive APIs and for the deep TLS check in Phase 7.
- **Scheduler:** either a loop inside the Node proxy or a Supabase Edge Function combined with pg_cron (no separate server required) that runs saved health-check requests on a real interval, independent of whether the phone app is open.
- **Health-check persistence:** write results to history with an is_health_check flag; compute and display uptime percentage and a latency trend per monitored endpoint.
- **Unattended soak testing:** reuse the same scheduler to run the soak mode from Phase 4 for durations longer than a phone can reliably sustain in the foreground.

Architecture Fit

Existing Piece	How It's Used / Extended
history table	Already the right shape to store health-check pings; only the is_health_check flag is new.
benchmarkEngine.ts soak mode (Phase 4)	The scheduler simply invokes the same soak logic server-side instead of on-device, removing the foreground/keep-awake constraint.

PHASE 11

CI / Export Capability

From "a tool I use" to "a tool my team relies on"

Lets a test suite built on mobile be exported and re-run in a CI pipeline — the feature that gives Axiom team-level value rather than individual-use value.

What to Build

- **Portable export format:** export a collection, its assertions, and its environment variables as a shareable JSON test-suite format.
- **Node CLI companion:** a small standalone runner that consumes the exported format and re-executes it outside the app. Because networkService.ts and assertionEngine.ts are plain TypeScript rather than React-Native-specific, most of this logic can be reused directly rather than rewritten.
- **CI-friendly output:** exit codes and a machine-readable pass/fail summary suitable for a pipeline step.

Architecture Fit

Existing Piece	How It's Used / Extended
networkService.ts, assertionEngine.ts, variableService.ts	All plain TS with no RN-only APIs beyond fetch(), which Node also supports — this is what makes a CLI companion realistic instead of a full rewrite.

Summary

Executed in sequence, these eleven phases take Axiom from a single-request tester with fixed-load benchmarking to a complete mobile QA suite: functional assertions and a chained collection runner form the foundation; environment matrix testing and advanced load modes extend what's already built; regression, contract, fuzz, security, chaos, and idempotency testing round out coverage of the major API testing categories; and the Node proxy scheduler plus CI export capability turn Axiom from an individual tool into one a team can depend on.

Each phase was scoped specifically against Axiom's existing codebase — networkService.ts, benchmarkEngine.ts, variableService.ts, environmentStore.ts, and the current Supabase schema — so implementation extends what already works rather than replacing it.